

The allegory axioms, and what defines them in the Frobenius calculus

Freyd, Categories, Algebras §2.11-§2.11.2 against functionalSemanticsForRelationalTheories.pdf Def. 4.1.
Lean: Freyd/S2_10.lean (Allegory).diag/CB.lean (CartBicat).diag/CB.Allegory.lean (allegoryOfCartBicat).
While the pictures come from Freyd, every picture of a theorem is omitted by scripts/diag_export from that theorem's Lean type. Nothing about a statement is redrawn here. The two figures marked transcribed are the exception: they copy the paper's definition pictures, and a definition has no statement to export.

Equation numbering. The numbers below are the 2017 preprint's. The CartBicat fields no longer encode them – a field is named for what it says. «RSet» rather than «RSet_27» – and each dotting carries the preprint number. The repo's copy of the paper is now the published version – Poly J. Math. 2 (6), 20 Oct 2025, doi:10.67903/polyjmath.2.6 – which remembers. Translation, published – preprint (34)–(37), (35)–(38), (39)–(39), (37)–(40), (38)–(41), (39)–(42), (40)–(43), (33)–(35), and Example 2.3(c)'s (13), (14)–(11), (12), the preprint's (34) is unnumbered there. Pages Def. 4.1 at 20–21 (was 17), the «Frobenius adj.1» (was 19), Lemma 4.2 at 21–22 (was 19). Remembering the Lean fields is a separate decision and has not been made.

Freyd's allegory takes reciprocity * and intersection n as primitive operations and lists eight equality axioms on them. A cartesian bicategory of relations takes each one either carries one special Frobenius structure Δ, 1, 7, the hom-order s is primitive, and both * and n are defined from the four dots. Every one of Freyd's eight axioms is then a theorem. The table below is that dictionary, one row per axiom.

Reading the pictures: left to right is composition – the book's diagram order, R S means first R then S – vertical stacking is s, a bare wire is an identity, a white box is a relation, and a box that is mirrored and tinted is its converse – the mirroring is the paper's s notation, the tint is this note's, because at a glance a chamber is easy to miss. The label inside is not mirrored: the paper can reverse a single letter and still be read, a whole term like R u S cannot. All four dots are solid, as Def. 4.1 draws them. The exporter draws a meet or a converse as a labelled box, so a composite under a * appears as a single box reading R ; S ; a draws as two stacked runs; and any subterm with no shape in this calculus – an associator, say – degrades to an opaque box. The term columns write the converse R*, Freyd's symbol, where the paper writes R†: these terms are read against the bouquet box, and one symbol per idea beats asking the reader to match two. The Lean sources write R* as well. The paper reserves (–)* for the colour swap of its §7, but §7 is a worked example in other models – see the caution at the end of this note – and nothing here formalises it, so in a repo that has Rel(Set) throughout there's no head to chop to protect against.

In a chain table the symbol at the head of each row is the relation that step was proved at = for a reshaping, or for a genuine inequality. Counting them is the point of drawing a proof at all – a chain of one s between equalities says the theorem is one law and a lot of bookkeeping.

1. Definition 4.1, as the paper states it
functionalSemanticsForRelationalTheories.pdf Def. 4.1, pp. 20–21 – transcribed here as class CartBicat (diag/CB.lean).field for field. Every picture below is exported from the Lean field, so the class and the definition cannot drift apart.

A cartesian bicategory of relations is a poset-enriched category, symmetric monoidal, satisfying four conditions. Clauses 1 and 2 are the two structures every object carries; clause 3 makes each the other's adjoint and imposes Frobenius; clause 4 is the only condition on arrows.

The tower this note describes is STRICT: the two bracketings of a threefold product are the same object, so no associator or unitor appears in any picture. The coherence arrows the paper suppresses in its own diagrams are absent here because they do not exist.

	(8) coassociativity CartBicat.Δ_assocC – copying twice on the left is copying twice on the right.
	(9) cocommutativity CartBicat.Δ_commC – the two copies are interchangeable.
	(10) count CartBicat.Δ_countC – copy, then discard one copy, and nothing has happened.

1.2. Every object carries a commutative monoid (v, ?)

	(5) associativity CartBicat.σ_assocC.
	(6) commutativity CartBicat.σ_commC.
	(7) unit CartBicat.σ_unitC – create, then merge, and nothing has happened.

	(37) ? ; Δ ≤ 1 – half of Δ + ?
	(38) 1 ≤ Δ ; ? – the other half
	(39) ? ; ! ≤ 1 – half of ! + ?
	(40) 1 ≤ ! ; ? – the other half
	(41) the Frobenius law CartBicat.frob_leftC, and its mirror frob_rightC. This is the one equation of the group: the other four are inequalities.

The special law Δ ; ? = 1 is NOT part of the definition. The paper derives it – “one direction is given by (38) and the other is proved as follows” – and so does the Lean, as Freyd.Diag.CartBicat.special:

1.4. Every arrow is a lax comonoid homomorphism

	(42) R ; Δ ≤ Δ ; (R ⊗ R) CartBicat.lax_Δ – running R once and copying s below copying and running it on both strands.
	(43) R ; ! ≤ ! CartBicat.lax_!.

Read as inequalities in one direction only: an arrow that satisfied these as EQUATIONS would be a map, and the picture is that a general relation is not.

2. Freyd's primitives are the Frobenius calculus's definitions

Freyd's primitive, and what defines it here	picture
reciprocal R* := R – §2.11.1 primitive. Here conv R := bend (R ⊗ 1) ; cap (diag/CB.lean:223), not a generator. Input arrives bottom left and runs under R; the cup ? ; Δ at the top left creates a pair, one strand feeding R and the other leaving as the output at the top right; the cap ? ; ! at the bottom right annihilates R's output against the input. ! is the cup and the cap that reverse the arrow – a wire newly bumped up over the box and back down is an isotopy of R itself.	
intersection R ∩ S := §2.11.1 primitive. Here the convolution meet R ∩ S := Δ ; (R ⊗ S) ; ∇ (diag/CB.Derived.lean:24). Copy the input, run both merges; the merge forces the two results to agree.	
containment R ⊆ S := §2.11.1 defined as R ∩ S = R. Here the post-entire containment is not mirrored, and n is derived, so the two directions are swapped. meet_eq_left_of_le (diag/CB.Derived.lean:196) is the bridge.	no picture – this is the s-cell itself
maximal morphism τ := §2.15, and Freyd needs a unitary allegory for it. Here top a b := ! ; ? (diag/CB.Derived.lean:228) is free in Def. 4.1. Freyd.Diag.«s_top».	

3. The eight axioms, and what proves each

Freyd's axiom, and what supplies it	picture, from the Lean statement
(R*)* = R – §2.11.1 recip_recip; here the snake, from (41). Freyd.Diag.CartBicat.conv_bowling down and back up straightens out, so bending twice is the identity.	
(R S)* = S* R* – §2.11.1 recip_comp; here mirroring the picture. Freyd.Diag.CartBicat.conv_reflecting a chain left to right reverses it; each box comes back flipped.	
(R ∩ S)* = R* ∩ S* – §2.11.1 recip_inter; the same mirroring. Freyd.Diag.conv_inter.Δ and ∇ are each other's mirror image, so reflecting the convolution leaves its shape alone and only turns the boxes round.	
R ∩ R = R – §2.11.1 inter_idem; here (42) and the special law. Freyd.Diag.meet_idem.Copy, run R twice, merge – (42) pushes R back through the copy and Δ ; ∇ = 1, closes the bubble.	
R ∩ S = S ∩ R – §2.11.1 inter_comm; here cocommutativity (9) and commutativity (6). Freyd.Diag.meet_comm.The two strands of Δ are interchangeable, and so are those of ∇.	
R ∩ (S ∩ T) = (R ∩ S) ∩ T – §2.11.1 inter_assoc; here coassociativity (8) and associativity (5). Freyd.Diag.meet_assoc.Either bracketing holds the same three-way copy tree, and dually the same merge tree.	
semi-distributivity R (S ∩ T) ⊆ (R ∩ S) ∩ T – §2.11.1 semidistrib; §2.136; here (42), the lax comonoid inequation. Freyd.Diag.semistrib_of_laxc.Running R and then copying s below copying and then running R each strand.	
modular identity R S ∩ T ⊆ (R ∩ S)* ∩ T – §2.11.2 modular, the axiom Freyd adjoins: “we adjoint one more containment, the law of modularity”. Here (41) and (42) derived. Freyd.Diag.modular_of_frobenius.Strip the shared prefix Δ ; (R ⊗ T) of both sides and what is left is «∇_slide_comm» (diag/CB.lean:569) – a box slides through a merge at the cost of its converse on the other strand.	

4. Unitarity and pre-tabular – the two conditions the converse needs

Freyd's §2.11 axioms are all a cartesian bicategory has to give back. Going the other way needs two further conditions, and they are exactly the two the repo has not formalised. Both definitions below are quoted from Freyd/S2_18.lean, which follows the book.

condition	Freyd	what the Frobenius side already has
unitary §2.189 R ∩ S = S ∩ R – §2.11.1 inter_idem; here (42) and the special law. Freyd.Diag.meet_idem.Copy, run R twice, merge – (42) pushes R back through the copy and Δ ; ∇ = 1, closes the bubble.	T is a partial unit when 1 T is the largest endomorphism on it, PartUnit T := ∇ ; R ; T → T ; R ⊆ 1 T (S.39); it is a unit when moreover every object carries an entire arrow to it, IsUnit T := PartUnit T A V a a, ∃ R : a → T, Entire R (S.545). A unitary Aℓlegory is an allegory with one (S.548). The point of a unit is that it makes a unit T_((a,b)) exist: t = p_0 p_0* through the unit, which is where §2.152's R p_0 ⊆ p_0* – the repo's (43) – comes from.	already there, for free. The monoidal unit 1 is the unit object, 1 ⊗ a → a is the entire arrow every object carries to it – (40) says so – and T* = 1 ; ? is maximal by Freyd.«s_top». So every cartesian bicategory of relations is unitary, with nothing assumed. Not recorded in Lean: aℓlegoryOfCartBicat builds a plain Aℓlegory, and no UnitaryAℓlegory instance is derived from CartBicat.
pre-tabular §2.165 f, g tabulate R when both are maps, R = f* g, and f* f ∩ g* g = 1 (Tabulates, S.502); R is tabular when some pair tabulates it (S.506). An allegory is PreTabularAℓlegory when every morphism is contained in a tabular one (S.556). §2.166. “If a unitary then pre-tabularity is equivalent to the existence of tabulations for maximal morphisms”.		this is what ø is. Given unitarity, the condition reduces to tabulating each τ_((a,b)), and a tabulation of τ_((a,b)) is precisely an object a ø b with its two projections – the monoidal product Def. 4.1 assumes. That is the whole content of the swap: Freyd derives the product from tabulations, Def. 4.1 posits it. Not built. Rebuilding from tabulations and products in Map(A) is cartBicatOfUnitaryPreTabular; diag/PLAN, and holds it back for want of a consumer.

Together they are the whole gap: cartesian bicategory of relations ø unitary pre-tabular allegory (diag/arnatiAlgebraOfFirstOrderLogic §10, after Carboni–Walters). One half of that equivalence is aℓlegoryOfCartBicat and is a theorem in this repo; the other half is the two rows above.

5. Union and residual – what phases 8 and 9 add

u and v are past where Freyd's §2.11 list stops, and past what Def. 4.1 can build: they need a biproduct on top of the Frobenius structure (diag/Tape.lean, after TapeDiagrams.pdf) and a second composition with linear adjoints (diag/F0.lean, after DiagrammaticAlgebraOfFirstOrderLogic.pdf). Both are a tape, so both draw.

The two notations say which layer a picture is in. An U is a tape, the rounded wrapper is the second monoidal product, and its s>/< open and close a branch a particle takes exactly one of – the same fork–runs–join shape as a meet, on a different product. A residual is a dashed box, because it is not a composite of the generators at all: R ∩ := R ; ∩ ; S1 lives in the second composition, and a solid box would claim it was drawable in the first.

statement	picture, from the Lean statement
R ⊆ R U S – §2.2 in Freyd; here Freyd.Diag.Biproduct.«union_left». union R S := (R ⊗ S) ; [1, 1] (diag/Tape.lean:84); offer both branches, then forget which was taken.	
R U S = S U R – Freyd.Diag.Biproduct.union_comm. The two branches of a tape are interchangeable, exactly as the two remainders of a dot:	
⊥ U R = R – Freyd.Diag.Biproduct.bot_union. bot a b := toZero a ; (toZero b) (diag/Tape.lean:87); routed through the zero object; the exporter has no shape for it, so it prints as a box.	
R (S U T) = (R ∩ S) U (R ∩ T) – Freyd.Diag.Biproduct.comp_union; composition distributes over union, which it does not. Freyd §2.2's distributivity, the reason a distributive allegory is more than an allegory.	
(R U S)* = R* U S* – Freyd.Diag.Biproduct.conv_union. The converse is a 2-functor for the tape layer too.	
R ∩ (S U T) = (R ∩ S) U (R ∩ T) – Freyd.Diag.FbCAlg.meet_union_distrib, the law that makes distributivityAℓlegoryOfFbCb a distributive allegory (§2.2). The one picture with both layers in it: it meets inside a tape on the left, tape around meets on the right.	
(R / S) ⊆ R – Freyd.Diag.ClosedLinearBicat.«residual_comp_s». Residuals is a lower bound of the arrows it is the greatest of. Freyd's §2.31 division: R / S is the largest T with T ⊆ R and Ue.«residual_iff» (diag/F0.lean:170) is that Galois to the full. The dashed frame marks the box as being uninterpretable in the second composition.	

6. Lemma 4.2 (ii) – the proof, not just the statement

diag_export --proof draws a proof. A cat chain elaborates to nested Trans, trans, so its intermediate terms are all recoverable from the proof term flattens the spine, infer the type of each fork, draw one cup per step. Nothing is replaced and nothing is retold – the nine rows below are the chain the Lean kernel accepted for Freyd.Diag.CartBicat.conv, which is Lemma 4.2 (ii). (R ; S)* = S* R*.

6.1. First: why these pictures are not pictures of a converse

They are not, and that is the thing to get straight before reading them. The converse is drawn with two bends – a cup on the way in and a cap on the way out, each a pair of dots, ? ; Δ and ? ; ! – which is the left-hand figure below. The proof never draws that. conv_unique says an arrow is the converse of R as soon as it satisfies one diagram from UNBENT arrows: a ø b – 1. The first bend is on the left, one cup on the right, bending is a bijection (bend_unbend/unbend_bend), so settling the unbent equation settles the box, and the cup never has to be drawn at all.

That is why every row below has a cap and no cup, and why an arrow that “is a converse” shows up not as a bent box but as a bent box sitting on the bottom strand, mirrored.

the converse, as the paper draws it	the one rule the proof uses, twice
R* is R with both wires bent round: in at the bottom left, cup at the top left, cap at the bottom right. Two dots at each bend, as functionalSemanticsForRelationalTheories.pdf draws it – a cup is ? then Δ, a cap is ? then !. Transcribed by hand; a definition has no statement to export.	conv_slide (diag/CB.lean:449): a box on the BOTTOM strand facing a cap is the same as that box on the TOP strand, upright. Sliding round the bend is what turns R* into R. This is the entire content of Lemma 4.2 (ii): the other seven steps are bookkeeping.

6.2. The chain

One row per term, not per step: the right-hand side of every step is the left-hand side of the next, so drawing steps would draw every picture twice. The first column is the term as the Lean has it, fully bracketed, with s binding tighter than > – it comes off the same walk as the picture, so the two cannot drift apart. Watch which strand each box is on: bottom means mirrored, top means upright.

Lemma 4.2 (ii), (R ; S)* = S* ; R*		the rule that reaches it
term 1 ø (S* ; R*) ; cap		the start. When conv_unique asks about S* ; R*, both boxes on the bottom strand, mirrored – that is what “these are converses” looks like once the wires are unbent.
= 1 ø S* ; (1 ø R*) ; cap		1. ø is a functor, so doing two things to one strand is sequence is the sequence of doing them. Same picture.
= 1 ø S* ; (R ø T) ; cap		2. conv_slide. R* is the box nearest the cap, so it slides round the bend: it leaves the bottom strand, arrives on the top, and comes back upright as R. The first of the two steps that prove anything.
= (1 ø S) ; R ø T ; cap		3. simp only [Cat.assoc]. Re-bracketing – visible in the term column, invisible in the picture. Same picture.
= R ø S* ; cap		4. tensor_split – interchange. R is on the top strand and S* on the bottom, so running them side by side is the same thing. Shows as a re-spacing only because the exporter lays a diagram out left to right.
= (R ø 1 ; 1 ø S*) ; cap		5. tensor_split – the same law, back the other way. Split them apart again, R first, so S* is now the box facing the cap, which is what the next step needs.
= R ø 1 ; (1 ø S*) ; cap		6. simp only [Cat.assoc]. Re-bracketing. Same picture.
= R ø 1 ; (S ø 1 ; cap)		7. conv_slide , on S and times. S* slides around the bend and arrives upright on the top strand. Both boxes are now on top, both upright, in the order R then S. The second and last real step.
= (R ø 1 ; S ø T) ; cap		8. simp only [Cat.assoc]. Re-bracketing. Same picture.
= (R ; S) ø 1 ; cap		9. tensor_comp – interchange. Since the two top-strand boxes merge into one. This is R ; S unbent, so conv_unique reads off S* R* = (R ; S)*. Done.

What the chain measures. Reads only pictures – and most consecutive pairs are the same drawing – steps 1, 3, 6, 8 and 9 change nothing at all, and step 4 changes only spacing – while the term column beside them changes at every step. These are associativity, the unit laws and interchange – precisely the laws a string diagram has not built in, and the theorem is two applications of Converse. Seven of these nine steps are bureaucracy: the picture language does change into it, and the theorem is two applications of Converse.

How this differs from the paper's proof. The paper proves (ii) by inserting a snake in the middle of R ; S and reading the two halves off as S* and R* – three pictures, all bent. The Lean never bends. Same theorem, different route, and the export shows the route the kernel accepted. Drawn by hand it would have shown the paper's three pictures and quietly implied those were what the repo had checked.

7. The modular law – proved, drawn

Freyd adjoins the modular identity: an allegory has no structure to derive it from, so it goes in as an axiom. Here it is a theorem, and this is its proof – Freyd.Diag.modular_of_frobenius (diag/CB.Allegory.lean:48), four terms, of which exactly one step is an inequality.

the modular law, R S ∩ T ≤ (R ∩ T)* S		the rule that reaches it
term (R ; S) ∩ T = Δ ; (R ø T) ; (S ø T) ; ∇		the start: R ∩ S ∩ T. A meet, so a copy and a merge with R S above and T below.
≤ Δ ; (R ø T) ; (S ø T) ; ∇		Factor out the shared prefix. (R S) ø T is (R ø T) ; (S ø T), so everything before the S is common to both sides and can be set aside. An equality, and drawn as one.
≤ Δ ; (R ø T) ; (1 ø S*) ; ∇		∇_slide_comm , under the shared prefix. S slides through the once the two top-strand boxes merge into one. This is R ø S unbent, so conv_unique reads off S* R* = (R ; S)*. Done.
= R ∩ T ; (S*) ; S		Fold the prefix back in. R ø (S*) is (R ø T) ; (1 ø S*), and the copy-meet pair reads as a meet again. (R ∩ T S*), S, which is the goal.

7.1. Inside the middle step

Def. 4.1 slide comm (diag/CB.Derived.lean:569) is where the mathematics is, and it mentions only S – R and T sit untouched in the prefix throughout. Read on its own it says one arrow slides through a merge.

$$\text{Freyd}.\text{Diag}.\text{entire_inter_iff}(\text{diag}/\text{S2}.\text{lean};213):-\text{Total}(\text{R } n \text{ S}) \rightarrow 1 \leq \text{R} ; \text{S}^*$$

Read the two sides: on the left $\text{R} \rightarrow \text{S}$ is named **twice**, on the right R and S are named **once each** and the meet is gone. That is the whole point of the lemma – the meet is needed to ask whether something is *entire*, not to **answer** it – and it is why every later rule uses the right-hand form. Freyd spends it at §2.147, where $H = x^* \vdash n^* y^*$ has to be shown *entire*; B&DM spend it throughout chapter 4.

Freyd. $\text{Diag}.\text{entire_inter_iff}(\text{ADP}/\text{Ad}.\text{2}.\text{lean};187)$ is the same statement for a bare allegory, and there it is $\text{Iff}.\text{rfl}$ on top of diag_inter in Freyd/§2.10. I mean the order $X \subseteq Y$ is **defined** as $x \wedge n^* = X$, so $\text{Dom}(\text{R } n \text{ S}) = 1^*$ and $1^* \subseteq \text{R} \text{ S}^*$ are the same proposition once §2.124 is known. Here $\leq$ is primitive, so both directions are real steps – and they cost very different things.

$$\begin{array}{c} \text{--- given R } n \text{ S entire, ---} \leq \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{---} \text{---} \\ \text{---} \text{---} \end{array} \begin{array}{c} \text{$$

What the associators cost. ø and p have no shape in this calculus and print as opaque boxes, so the two middle terms of the inner chain are less legible than c's and t's. That is the honest price of drawing what the Lean says: the reshaping steps really are that fiddly, and a hand-drawn version would have hidden them by working up to coherence.

8. Maps: SV, TOT, INJ, SUR, and the two inequations that always hold

Def. 4.1 clause 4 says every arrow is lax for A and for ! – four inequations (42) and (43). Applying Lemma 4.2 (ii) and (iv) to them gives (44) and (45), the same statements about ∇ and ? . They force ! holds for every arrow. Their four REVERSES do not, and each one failing or holding is a property of the arrow: that is (SV), (TOT), (INJ), (SUR).

	holds for every arrow	holds iff the arrow is ...
copy Δ		(SV) – single valued
discard !		(TOT) – total
merge ∇		(INJ) – injective
unit ?		(SUR) – surjective

The paper gives the same four properties a second time, as ADJOINT conditions – (46)–(49) – and Lemma 4.4 says the two presentations agree: diag/CB.Derived.lean takes the adjoint forms as the definitions, because they are literally Freyd's Simple and Entire (Def. 4.13), so a diag map and a book map are one predicate with no translation layer.

property	adjoint form, the definition	Freyd
single valued Freyd.Diag.SingleValued		Simple R := R* R ⊆ 1 §2.13
total Freyd.Diag.Total		Entire R := 1 ⊆ R R* §2.13. With single valued, R is a map.
injective Freyd.Diag.Injective		R R* ⊆ 1 §2.141 at f = g = R : a monic pair, so a monic map.
surjective Freyd.Diag.Surjective		§2.147: “g is a cover iff 1 ⊆ g* g”, that is, iff g* is entire.

Why (44) and (45) have no picture. Both are converse-inequalities, so drawing them from a Lean statement needs that statement to exist, and it does not. Getting it needs A* = ∇, !* = ? and Lemma 4.2 (iii) (f ø g)* = f* g* – and the first of these is a Frobenius computation at a COMPOSITE object, which is the clause of Carboni & Walters' definition that Def. 4.1 as printed omits (diag/CB.lean's header records this). It is the same gap that leaves Lemma 4.4 unproved in the repo. The four reverse inequations opposite are unaffected: they are recorded under the paper's own laws in diag/CB.Derived.lean, and nothing downstream depends on Lemma 4.4.

9. Entireness of an intersection

Two definitions from Freyd, both below. The domain of R is the part of the identity where R is defined, Dom R = 1 ∩ R R* (§2.122). R is entire iff it is defined everywhere. Dom R = 1, equivalently 1 ⊆ R R* (§2.13) – which is Total in diag/CB.Derived.lean – is the same statement, so that there is a Lean declaration in diag/terms for the exporter to draw. Nothing new is assumed: both are proved from Map alone.

Each rule is an s, so each is two chains. The statement of a branch is its header: the hypothesis it is given and the goal it reaches, both drawn from the same Lean – and the chain proves.

The proofs, drawn. Each is a two chains, and diag_export --proof now splits an Ifff.intro into its two branches rather than drawing the longer one and dropping the other. The point to watch is that each direction spends exactly one half of Map – never both.

shunting right, == given		the rule that reaches it the start: R f.
term R		the hypothesis, met with 1. meet_eq_left 1 ≤ 1 and 1 ≤ R S*, so 1 is before the meet of the two.
≤ R ; (f* ; f*)		§2.124: 1 ∩ R S* = 1 ∩ (S ∩ R) (S ∩ R)*. R S* relates to S* when some b has a R b and a S* b. The f is the same as S*, and a and b then reads “some b has a R b and a S* b” – one arrow of R ∩ S. Two arrows become one: that is the step, and it is what needs the modular law.
≤ S ; f*		Freyd writes the right-hand side Dom (S ∩ R) as Dom S ∩ Dom R, his name for “1 ∩ R S*” is the same term, and an extra operation.
shunting right, == given		X ∩ Y ≤ Y, then S ∩ R = R ∩ S. The outer fork drops away, leaving the goal.

What the asymmetry says. One direction is two applications of monot